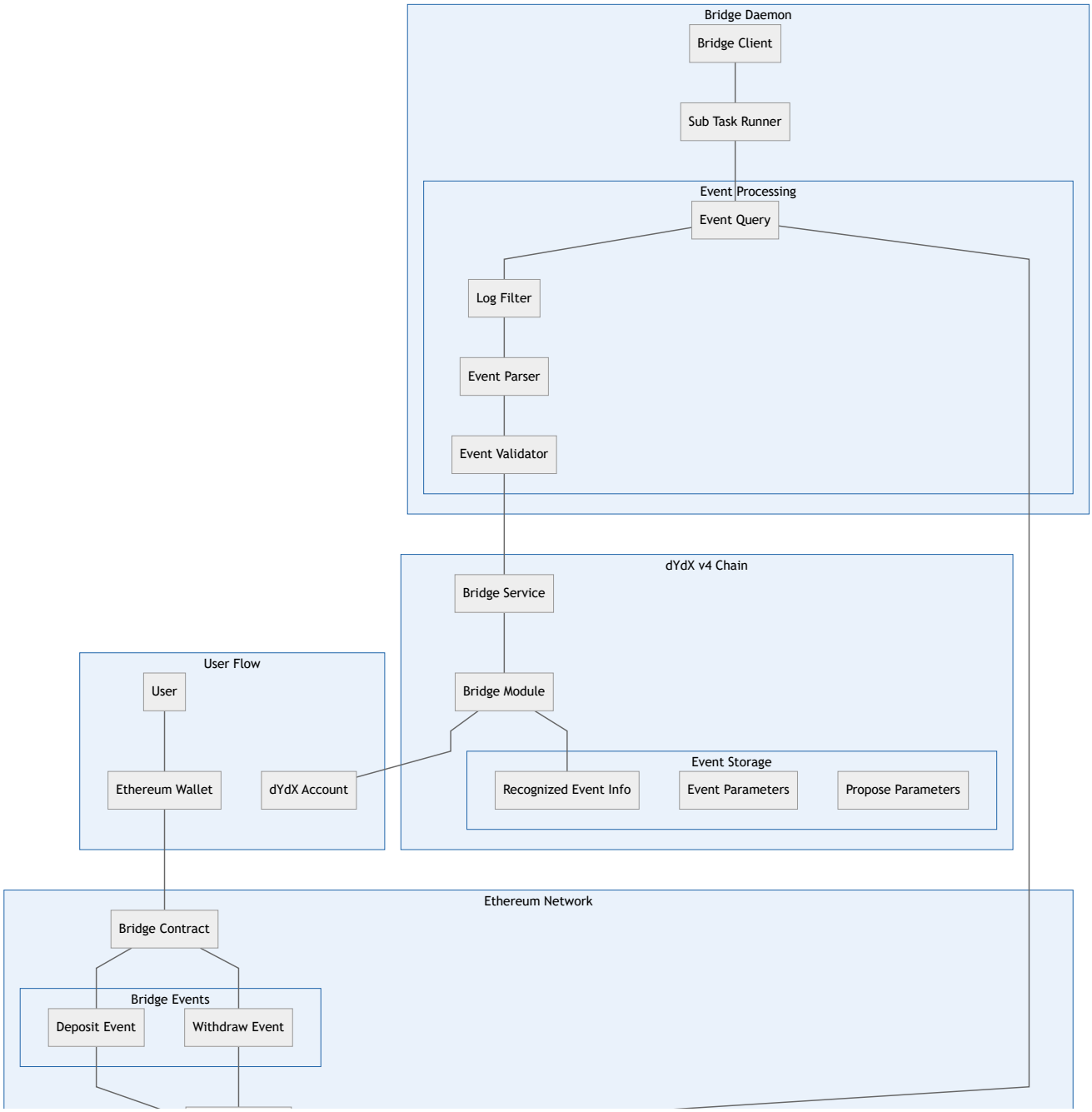
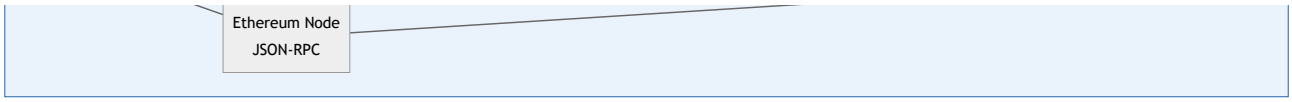
Bridge Daemon 구현 분석

개요

Bridge Daemon은 dYdX v4 Protocol의 크로스체인 자산 이동을 담당하는 핵심 구성 요소로, 이더리움 블록체인에서 발생하는 브릿지 이벤트를 모니터링하고 이를 dYdX v4 체인으로 중계하는 역할을 수행합니다. 이 데몬은 이더리움 브릿지 컨트랙트의 이벤트를 실시간으로 감지하여 사용자의 입금 및 출금 요청을 안전하게 처리합니다.

크로스체인 브릿지 아키텍처


Bridge Daemon
Bridge Client
Sub Task Runner
Event Processing
Event Query
Log Filter
Event Parser
Event Validator
dYdX v4 Chain
Bridge Service
Bridge Module
Event Storage
Recognized Event Info
Event Parameters
Propose Parameters
User Flow
User
Ethereum Wallet
dYdX Account
Ethereum Network
Bridge Contract
Bridge Events
Deposit Event
Withdraw Event



브릿지 아키텍처 구성 요소

1. 이더리움 브릿지 컨트랙트

- **역할**: 사용자의 자산을 잠그고 해제하는 스마트 컨트랙트
- **주요 기능**:
- 사용자 자산 예치 (Deposit)
- 자산 출금 승인 (Withdrawal Authorization)
- 브릿지 이벤트 발생

2. Bridge Daemon

- **역할**: 이더리움 이벤트를 모니터링하고 dYdX 체인으로 중계
- **주요 기능**:
- 이더리움 노드 연결 및 이벤트 조회
- 이벤트 파싱 및 검증
- dYdX 체인으로 이벤트 전송

3. dYdX Bridge Module

- **역할**: 브릿지 이벤트를 처리하고 사용자 계정에 자산 반영
- **주요 기능**:
- 이벤트 검증 및 저장
- 사용자 계정 잔고 업데이트
- 중복 처리 방지

이더리움 브릿지 이벤트 모니터링 구현

1. 이벤트 모니터링 메커니즘

주기적 이벤트 조회

```
func (c *Client) Start(ctx context.Context,
flags daemonflags.DaemonFlags, ...) error {

// 이더리움 RPC 엔드포인트 연결

ethClient, err := ethclient.Dial(flags.Bridge.EthRpcEndpoint)

if err != nil {

return fmt.Errorf("failed to establish connection to Ethereum
node: %w", err)

}

defer ethClient.Close()

// 주기적 실행을 위한 티커 설정

ticker := time.NewTicker(time.Duration(flags.Bridge.LoopDelayMs)
* time.Millisecond)

stop := make(chan bool, 1)
```

```
// 메인 태스크 루프 시작
StartBridgeDaemonTaskLoop(
ctx,
c,
ticker,
stop,
&SubTaskRunnerImpl{},
ethClient,
queryClient,
serviceClient,
)
return nil
}
```

이벤트 조회 태스크 루프

```
func StartBridgeDaemonTaskLoop(
ctx context.Context,
c *Client,
ticker *time.Ticker,
stop <-chan bool,
s SubTaskRunner,
ethClient types.EthClient,
queryClient bridgetypes.QueryClient,
```

```
serviceClient api.BridgeServiceClient,
) {
for {
select {
case <-ticker.C:
// 주기적으로 브릿지 이벤트 처리 실행
if err := s.RunBridgeDaemonTaskLoop(
ctx,
c.logger,
ethClient,
queryClient,
serviceClient,
); err == nil {
c.ReportSuccess()
} else {
c.logger.Error("Bridge daemon returned error", "error", err)
c.ReportFailure(err)
}
case <-stop:
return
}
}
```

```
}
```

2. 이더리움 이벤트 필터링

브릿지 이벤트 필터 쿼리 생성

```
func getFilterQuery(

contractAddressHex string,

fromBlock uint64,

firstId uint32,

numIds uint32,

) eth.FilterQuery {

// 다음 numIds개의 이벤트 ID에 대한 해시 생성

eventIdHashes := make([]ethcommon.Hash, numIds)

for i := uint32(0); i < numIds; i++ {

eventIdHashes[i] = ethcommon.BigToHash(new(big.Int)
.SetUint64(uint64(firstId + i)))

}

return eth.FilterQuery{

FromBlock: new(big.Int).SetUint64(fromBlock),

ToBlock: big.NewInt(ethrpc.FinalizedBlockNumber.Int64()),
// 최종 확정된 블록까지

Addresses: []ethcommon.Address{ethcommon.HexToAddress(contractAddressHex)},

Topics: [][]ethcommon.Hash{

{ethcommon.HexToHash(constants.BridgeEventSignature)},
// Bridge 이벤트 시그니처
```

```
eventIdHashes, // 특정 이벤트 ID들만 필터링
},
}
}
```

브릿지 이벤트 시그니처 및 ABI

```
const (
// Bridge 이벤트 시그니처: sha256("Bridge(uint256,uint256,address,bytes,bytes)")
BridgeEventSignature = "0x498a04382650bc110983392ed12ab27595af8ece270a344fc70d77
// Bridge 이벤트 ABI 정의
BridgeEventABI = `[
{
"anonymous":false,
"inputs":[
{
"indexed":true,
"internalType":"uint256",
"name":"id",
"type":"uint256"
},
{
"indexed":false,
```

"internalType":"uint256",

"name":"amount",

"type":"uint256"

},

{

"indexed":false,

"internalType":"address",

"name":"from",

"type":"address"

},

{

"indexed":false,

"internalType":"bytes",

"name":"accAddress",

"type":"bytes"

},

{

"indexed":false,

"internalType":"bytes",

"name":"data",

"type":"bytes"

}

],

```
"name":"Bridge",

"type":"event"

}

]`

)
```

3. 이벤트 조회 및 처리 프로세스

메인 태스크 루프 구현

```
func (s *SubTaskRunnerImpl) RunBridgeDaemonTaskLoop(

ctx context.Context,

logger log.Logger,

ethClient types.EthClient,

queryClient bridgetypes.QueryClient,

serviceClient api.BridgeServiceClient,

) error {

// 1. Bridge 모듈에서 설정 정보 조회

eventParams, err := queryClient.EventParams(ctx, &bridgetypes.QueryEventParamsRe

if err != nil {

return fmt.Errorf("failed to fetch event params: %w", err)

}

proposeParams, err := queryClient.ProposeParams(ctx, &bridgetypes.QueryProposePa

if err != nil {
```

```
return fmt.Errorf("failed to fetch propose params: %w", err)
}
recognizedEventInfo, err := queryClient.RecognizedEventInfo(ctx, &bridgetypes.Qu
if err != nil {
return fmt.Errorf("failed to fetch recognized event info: %w", err)
}
// 2. 이더리움 체인 ID 검증
chainId, err := ethClient.ChainID(ctx)
if err != nil {
return fmt.Errorf("failed to fetch chain ID: %w", err)
}
if chainId.Uint64() != eventParams.Params.EthChainId {
return fmt.Errorf(
"expected chain ID %d but node has chain ID %d",
eventParams.Params.EthChainId,
chainId,
)
}
// 3. 이더리움 노드에서 로그 조회
filterQuery := getFilterQuery(
eventParams.Params.EthAddress,
recognizedEventInfo.Info.EthBlockHeight,
recognizedEventInfo.Info.NextId,
```

```
proposeParams.Params.MaxBridgesPerBlock,
)
logs, err := ethClient.FilterLogs(ctx, filterQuery)
if err != nil {
return fmt.Errorf("failed to fetch logs: %w", err)
}
// 4. 로그를 브릿지 이벤트로 파싱
newBridgeEvents := make([]bridgetypes.BridgeEvent, len(logs))
for i, log := range logs {
newBridgeEvents[i] = libeth.BridgeLogToEvent(log, eventParams.Params.Denom)
}
// 5. 브릿지 서버로 이벤트 전송
if _, err = serviceClient.AddBridgeEvents(ctx, &api.AddBridgeEventsRequest{
BridgeEvents: newBridgeEvents,
}); err != nil {
return fmt.Errorf("failed to add bridge events: %w", err)
}
return nil
}
```

입출금 처리 및 검증 프로세스

1. 이더리움 로그 파싱

브릿지 이벤트 변환

```
func BridgeLogToEvent(log ethcoretypes.Log, denom string) bridgetypes.BridgeEven

// 1. 토픽에서 이벤트 ID 추출 (indexed 필드)

id := lib.MustConvertIntegerToUint32(log.Topics[1].Big().Uint64())

// 2. 데이터에서 이벤트 정보 언팩

bridgeEventData, err := GetBridgeEventAbi().Unpack("Bridge", log.Data)

if err != nil {

panic(err)

}

amount := bridgeEventData[0].(*big.Int) // 브릿지 금액

fromAddress := bridgeEventData[1].(ethcommon.Address) // 송신자 이더리움 주소

toAddress := PadOrTruncateAddress(bridgeEventData[2].([]byte)) // 수신자 dYdX 주소

memo := bridgeEventData[3].([]byte) // 사용자 메모

// 3. dYdX 브릿지 이벤트 구조체 생성

return bridgetypes.BridgeEvent{

Id: id,

Coin: sdk.NewCoin(denom, sdkmath.NewIntFromBigInt(amount)),

Address: sdk.MustBech32ifyAddressBytes(config.Bech32PrefixAccAddr, toAddress),

EthBlockHeight: log.BlockNumber,

}
```

```
}
```

주소 형식 변환

```
func PadOrTruncateAddress(address []byte) []byte {
// dYdX 주소 형식: [prefix][separator][address][checksum]
// - prefix: "dydx"
// - separator: "1"
// - address: 실제 주소 부분 (20바이트)
// - checksum: 마지막 6자리
const expectedLength = 20
if len(address) == expectedLength {
return address
}
result := make([]byte, expectedLength)
if len(address) > expectedLength {
// 길이가 초과하는 경우 뒤에서부터 20바이트 추출
copy(result, address[len(address)-expectedLength:])
} else {
// 길이가 부족한 경우 앞쪽을 0으로 패딩
copy(result[expectedLength-len(address):], address)
}
return result
```

```
}
```

2. 브릿지 이벤트 구조

BridgeEvent 데이터 구조

```
type BridgeEvent struct {
// 이더리움 이벤트 로그의 고유 ID
Id uint32 `protobuf:"varint,1,opt,name=id,proto3" json:"id,omitempty"`
// 브릿지된 토큰 (금액과 denomination)
Coin types.Coin `protobuf:"bytes,2,opt,name=coin,proto3" json:"coin"`
// 브릿지 대상 계정 주소 또는 모듈 주소
Address string `protobuf:"bytes,3,opt,name=address,proto3" json:"address,omitemp
// 이벤트가 발생한 이더리움 블록 높이
EthBlockHeight uint64 `protobuf:"varint,4,opt,name=eth_block_height,json=ethBloc
}
```

브릿지 파라미터 설정

```
type EventParams struct {
// 이더리움 체인 ID (메인넷: 1, 테스트넷: 5)
EthChainId uint64 `protobuf:"varint,1,opt,name=eth_chain_id,json=ethChainId,prot
// 브릿지 컨트랙트 주소
EthAddress string `protobuf:"bytes,2,opt,name=eth_address,json=ethAddress,proto3
```

```
// 브릿지되는 토큰의 denomination (예: "usdc")

Denom string `protobuf:"bytes,3,opt,name=denom,proto3" json:"denom,omitempty"`

}



type ProposeParams struct {

// 블록당 최대 브릿지 이벤트 수

MaxBridgesPerBlock uint32 `protobuf:"varint,1,opt,name=max_bridges_per_block,jso

// 브릿지 이벤트 제안 지연 시간

ProposeDelayDuration time.Duration `protobuf:"bytes,2,opt,name=propose_delay_dur

// 브릿지 이벤트 건너뛰기 지연 시간

SkipDelayDuration time.Duration `protobuf:"bytes,3,opt,name=skip_delay_duration,

// 브릿지 이벤트 건너뛰기 비율 (ppm 단위)

SkipRatePpm uint32 `protobuf:"varint,4,opt,name=skip_rate_ppm,json=skipRatePpm,p

}
```

3. 이벤트 전송 및 처리

gRPC를 통한 이벤트 전송

```
func (s *SubTaskRunnerImpl) sendBridgeEvents(

ctx context.Context,

serviceClient api.BridgeServiceClient,

bridgeEvents []bridgetypes.BridgeEvent,

) error {
```

```
// 브릿지 이벤트 추가 요청 생성
request := &api.AddBridgeEventsRequest{
BridgeEvents: bridgeEvents,
}
// Bridge Service로 이벤트 전송
response, err := serviceClient.AddBridgeEvents(ctx, request)
if err != nil {
return fmt.Errorf("failed to add bridge events: %w", err)
}
// 성공적으로 처리된 이벤트 수 로깅
logger.Info(
"Successfully sent bridge events",
"count", len(bridgeEvents),
"response", response,
)
return nil
}
```

이벤트 순서 보장

```
type RecognizedEventInfo struct {
// 다음 처리할 이벤트 ID (마지막 처리된 ID + 1)
NextId uint32 `protobuf:"varint,1,opt,name=next_id,json=nextId,proto3" json:"nex
```

```
// 이벤트 조회를 시작할 이더리움 블록 높이

EthBlockHeight uint64 `protobuf:"varint,2,opt,name=eth_block_height,json=ethBloc

}



// 순차적 이벤트 ID 검증

func validateEventSequence(events []bridgetypes.BridgeEvent, expectedNextId uint

for i, event := range events {

expectedId := expectedNextId + uint32(i)

if event.Id != expectedId {

return fmt.Errorf(

"event ID mismatch: expected %d, got %d at index %d",

expectedId,

event.Id,

i,

)

}

}

return nil

}
```

크로스체인 보안 메커니즘

1. 이벤트 검증 및 무결성 보장

체인 ID 검증

```go
func (s *SubTaskRunnerImpl) validateChainId(
ctx context.Context,
ethClient types.EthClient,
expectedChainId uint64,
) error {
// 이더리움 노드에서 체인 ID 조회
chainId, err := ethClient.ChainID(ctx)
if err != nil {
return fmt.Errorf("failed to fetch chain ID: %w", err)
}
// 예상 체인 ID와 비교
if chainId.Uint64() != expectedChainId {
return fmt.Errorf(
"chain ID mismatch: expected %d, got %d",
expectedChainId,
chainId.Uint64(),
)
}
return nil
}
```

컨트랙트 주소 검증

```
func validateContractAddress(eventParams *bridgetypes.EventParams) error {
// 브릿지 컨트랙트 주소가 올바른 형식인지 검증
if !ethcommon.IsHexAddress(eventParams.EthAddress) {
return fmt.Errorf("invalid contract address format: %s", eventParams.EthAddress)
}
// 제로 주소가 아닌지 확인
if eventParams.EthAddress == ethcommon.HexToAddress("0x0").Hex() {
return fmt.Errorf("contract address cannot be zero address")
}
return nil
}
```

2. 최종성 보장 (Finality)

최종 확정된 블록만 조회

```
func getFilterQuery(contractAddressHex string, fromBlock uint64, firstId uint32,
return eth.FilterQuery{
FromBlock: new(big.Int).SetUint64(fromBlock),
// 최종 확정된 블록까지만 조회하여 재구성 공격 방지
ToBlock: big.NewInt(ethrpc.FinalizedBlockNumber.Int64()),
Addresses: []ethcommon.Address{ethcommon.HexToAddress(contractAddressHex)},
Topics: [][]ethcommon.Hash{
```

```
{ethcommon.HexToHash(constants.BridgeEventSignature)},

eventIdHashes,

},

}

}
```

블록 확정성 대기

```
const (

// 이더리움 네트워크에서 블록 최종성을 위한 최소 확인 수

MinConfirmations = 12

// 최종성 대기 시간 (약 2.5분)

FinalityWaitTime = 150 * time.Second

)


func waitForFinality(ctx context.Context, ethClient types.EthClient, targetBlock

for {

currentBlock, err := ethClient.BlockNumber(ctx)

if err != nil {

return fmt.Errorf("failed to get current block number: %w", err)

}

// 충분한 확인 수가 쌓였는지 확인

if currentBlock >= targetBlockHeight+MinConfirmations {
```

```
return nil

}

// 잠시 대기 후 재확인

select {

case <-ctx.Done():

return ctx.Err()

case <-time.After(10 * time.Second):

continue

}

}

}
```

3. 중복 처리 방지

이벤트 ID 기반 중복 방지

```
type BridgeEventTracker struct {

processedEvents map[uint32]bool

mutex sync.RWMutex

}


func (t *BridgeEventTracker) IsProcessed(eventId uint32) bool {

t.mutex.RLock()

defer t.mutex.RUnlock()
```

```
return t.processedEvents[eventId]
}

func (t *BridgeEventTracker) MarkProcessed(eventId uint32) {
t.mutex.Lock()
defer t.mutex.Unlock()
t.processedEvents[eventId] = true
}

func (s *SubTaskRunnerImpl) filterDuplicateEvents(
events []bridgetypes.BridgeEvent,
tracker *BridgeEventTracker,
) []bridgetypes.BridgeEvent {
var newEvents []bridgetypes.BridgeEvent
for _, event := range events {
if !tracker.IsProcessed(event.Id) {
newEvents = append(newEvents, event)
tracker.MarkProcessed(event.Id)
}
}
return newEvents
}
```

원자적 이벤트 처리

```
func (s *SubTaskRunnerImpl) processEventsAtomically(
ctx context.Context,
serviceClient api.BridgeServiceClient,
events []bridgetypes.BridgeEvent,
) error {
// 트랜잭션 시작
tx, err := serviceClient.BeginTransaction(ctx)
if err != nil {
return fmt.Errorf("failed to begin transaction: %w", err)
}
defer func() {
if err != nil {
// 에러 발생 시 롤백
if rollbackErr := tx.Rollback(); rollbackErr != nil {
logger.Error("Failed to rollback transaction", "error", rollbackErr)
}
}
}()
// 모든 이벤트를 원자적으로 처리
for _, event := range events {
if err = s.processEvent(ctx, tx, event); err != nil {
```

```
return fmt.Errorf("failed to process event %d: %w", event.Id, err)

}

}

// 트랜잭션 커밋

if err = tx.Commit(); err != nil {

return fmt.Errorf("failed to commit transaction: %w", err)

}

return nil

}
```

4. 에러 처리 및 복구

연결 실패 시 재시도 로직

```
func (c *Client) connectWithRetry(ctx context.Context, endpoint string, maxRetri

var client *ethclient.Client

var err error

for attempt := 0; attempt < maxRetries; attempt++ {

client, err = ethclient.Dial(endpoint)

if err == nil {

return client, nil

}

// 지수 백오프로 재시도

backoffDuration := time.Duration(1<<attempt) * time.Second
```

```
c.logger.Warn(
"Failed to connect to Ethereum node, retrying",
"attempt", attempt+1,
"maxRetries", maxRetries,
"backoff", backoffDuration,
"error", err,
)
select {
case <-ctx.Done():
return nil, ctx.Err()
case <-time.After(backoffDuration):
continue
}
}
return nil, fmt.Errorf("failed to connect after %d attempts: %w", maxRetries, er
}
```

부분 실패 처리

```
func (s *SubTaskRunnerImpl) handlePartialFailure(
ctx context.Context,
events []bridgetypes.BridgeEvent,
failedIndex int,
```

```
err error,

) error {

// 실패한 이벤트 이전까지는 성공적으로 처리된 것으로 간주

successfulEvents := events[:failedIndex]

failedEvents := events[failedIndex:]

s.logger.Error(

"Partial failure in event processing",

"successfulCount", len(successfulEvents),

"failedCount", len(failedEvents),

"firstFailedEventId", failedEvents[0].Id,

"error", err,

)

// 실패한 이벤트들을 재처리 큐에 추가

return s.requeueFailedEvents(ctx, failedEvents)

}
```

결론

Bridge Daemon은 dYdX v4의 크로스체인 자산 이동을 안전하고 효율적으로 처리하는 핵심 구성 요소로서 다음과 같은 주요 특징을 가집니다:

1. **실시간 이벤트 모니터링**: 이더리움 블록체인의 브릿지 이벤트를 실시간으로 감지하고 처리
2. **강력한 보안 메커니즘**: 체인 ID 검증, 최종성 보장, 중복 처리 방지를 통한 안전한 크로스체인 전송
3. **순차적 이벤트 처리**: 이벤트 ID 기반의 순서 보장으로 데이터 무결성 확보

4. **견고한 에러 처리**: 연결 실패, 부분 실패 등 다양한 오류 상황에 대한 복구 메커니즘

5. **효율적인 필터링**: 스마트 컨트랙트 이벤트 시그니처와 토픽 필터링을 통한 정확한 이벤트 수집

이러한 설계를 통해 Bridge Daemon은 사용자가 이더리움과 dYdX v4 체인 간에 안전하고 신뢰할 수 있는 자산 이동을 수행할 수 있도록 지원합니다.